

Полиморфизмът е способността на даден обект да приема повече от една форма.

Подтипов полиморфизъм и виртуални функции

При наследяване например `class Dog: public Animal`, `Dog` е подтип на `Animal` и обектите от тип `Dog` могат да се разглеждат като `Animal`.

При извикване на метод има два начина по които се издирва имплементация

- статично свързване - конкретната имплементация се издирва по време на компилация
- динамично свързване - конкретната имплементация се издирва по време на изпълнение, но трябва метода да е маркиран с `virtual`

```
class Wolf {  
public:
```

```
    virtual void makeSound() const {
```

```
        std::cout << "woof";
```

```
    }
```

```
}
```

```
Dog dog;
```

```
Wolf* wolf = &dog;
```

```
wolf->makeSound() // ще се извика makeSound на Dog
```

```
class Dog: public Wolf {
```

```
public:
```

```
    void makeSound() const {
```

```
        std::cout << "bau";
```

```
    }
```

```
}
```

В този пример показваме динамично свързване, но ако `makeSound` в `Wolf` не беше маркиран като `virtual`, то щеше да се извика `makeSound` на `Wolf` (статично свързване). Хубаво е в последните методи които ще използваме полиморфично да се анотират с `override`, например в `Dog`

```
void makeSound() const override {
```

```
    std::cout << "bau";
```

```
}
```

така ако `makeSound` не е `virtual` в `Wolf` или сигнатурата се различава, ще имаме грешка още по време на компилация. Достатъчно е функцията да бъде обявена за `virtual` само в основния клас, от там нагоре.

близки наследници които я override-ват с virtual.

Абстрактни класове и методи

Понякога може да няма смисъл да даваме имплементация на метод в основен клас, който обаче искаме наследниците да имат, тогава може метода да се направи абстрактен като се маркира като virtual и след сигнатурата пишем =0;. Клас с поне 1 такъв метод се нарича абстрактен и не може да имаме обекти от този клас.

```
class Animal {
```

```
public:
```

```
    virtual void makeSound() const =0; // makeSound() няма смисъл за generic Animal
```

```
};
```

По този начин наследниците трябва или да дадат имплементация на makeSound() или да го маркират като абстрактен, но това и този наследник ще е абстрактен. Важно е да се каже, че въпреки че не може да имаме обект от тип Animal (защото е абстрактен), Animal* е възможно, т.е. Animal* animal = new Dog() е валидно, стига Dog да не е абстрактен. Ако близки методи в класа са виртуални и класът няма публични данни, то такъв клас се нарича интерфейс.

Друго много важно нещо е, че в йерархия предназначена за полиморфно ползване трябва деструктора на корена да е се маркира като virtual за да няма memory leaks.

```
class Animal {
```

```
public:
```

```
    virtual void makeSound() const =0;
```

```
    virtual ~Animal() {}
```

```
};
```

```
class Wolf: public Animal {
```

```
public:
```

```
    void makeSound() const override {
```

```
        std::cout << "woof";
```

```
    }
```

```
};
```

```
class Dog: public Wolf {
```

```
public:
```

```
void makeSound() const override {
```

```
    std::cout << "bau";
```

```
}
```

```
};
```

По този начин ако имаме `Animal* animal = new Dog();` и `delete animal;` ще се извика `~Dog()`, `~Wolf()` и накрая `~Animal()`

Поневе масивите са крайни редици от елементи от един и същ тип не можем да имаме на една позиция обект от клас `Wolf`, а на друга - обект от тип `Dog`. Вместо това можем да направим масив от указатели към базов клас, който могат да държат адрес на обект от клас наследник на базовия. Пример (ползваме иерархията от горния пример)

```
Wolf wolf;
```

```
Dog dog;
```

```
Animal* animals[] = {&dog, &wolf};
```

```
for (int i=0; i<z; ++i) {
```

```
    animals[i] -> makeSound();
```

```
}
```

За всеки елемент в масива ще се извика `makeSound()` за конкретния клас.

Параметризен полиморфизъм

Параметризният полиморфизъм дава една имплементация за произволни типове.

За целта ползваме шаблони

Шаблона на функција

```
template <typename T>
```

```
void swap(T& x, T& y) {
```

```
    T temp = x;
```

```
    x = y;
```

```
    y = temp;
```

```
}
```

```
int a = 1;
```

```
int b = 2;
```

```
char c = 'c';
```

```
char d = 'd';
```

```
swap(a, b); // работи
```

```
swap(c, d); // работи
```

```
swap(a, d) // не работи, затоа што во шаблонот и дојде со тип T, т.е. со int во нив
```

За всяко извиквање компилаторот генерира версија на функцијата со конкретни типови.

```
template <typename T>
```

```
class Box {
```

```
    T value;
```

```
public:
```

```
    Box(const T& value): value(value) {}
```

```
    T get() const { return value; }
```

```
    void set(const T& value);
```

```
};
```

```
template <typename T>
```

```
void Box<T>::set(const T& value) {
```

```
    this->value = value;
```

```
}
```

Можемо и да дефинираме и извршен клас (примера горе `Box<T>::set()`)

В горния пример Box е абстрактен клас и може да се използва по следния начин:

```
Box<int> box(5);
```

```
Box<double> box2(3.0);
```

```
Box<std::string> box3("pesha");
```

Множествено наследяване

В C++ произведем клас който да наследява повече от един клас:

```
class Person {
```

```
    std::string name;
```

```
public:
```

```
    Person(const std::string& name): name(name) {}
```

```
};
```

```
class Student: virtual public Person {
```

```
    int fn;
```

```
public:
```

```
    Student(const std::string& name, int fn): Person(name), fn(fn) {}
```

```
    void print() const {
```

```
        std::cout << fn;
```

```
    }
```

```
};
```

```
class Worker: virtual public Person {
```

```
    double salary;
```

```
public:
```

```
    Worker(const std::string& name, double salary): Person(name), salary(salary) {}
```

```
    void print() const {
```

```
        std::cout << salary;
```

```
    }
```

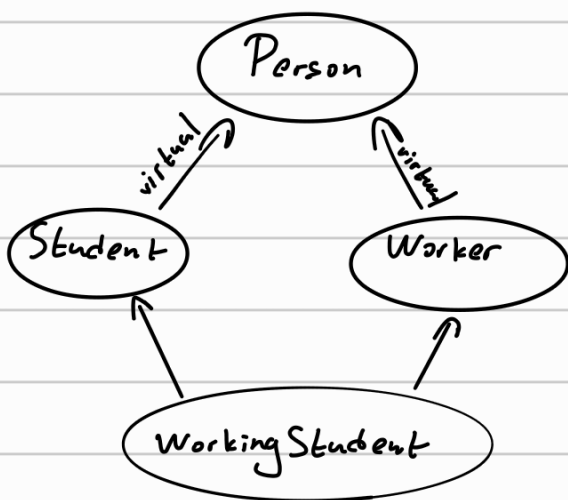
```
};
```

```

class WorkingStudent: public Worker, public Student {
public:
    WorkingStudent(const std::string& name, int fn, double salary):
        Person(name), Student(name, fn), Worker(salary) {}
};

```

В този пример WorkingStudent наследява Student и Worker. Важно е да се отбележи, че Student и Worker наследяват Person виртуално, т.е. `class Student: virtual public Person` и `class Worker: virtual public Person`. Ако нямаме виртуално наследяване WorkingStudent няма да наследява пряко Person - едно от Worker и едно от Student и `Person* person = &workingStudent` ще даде грешка, защото нямаме нееднозначност. Това се решава с виртуално наследяване (както беше с направено в примера).



Има и още един проблем с множествено наследяване. Дори с виртуално наследяване ако имаме WorkingStudent обект, който извика `print()` това да даде грешка, защото `print` е нееднозначно (едно от Student и едно от Worker). Това може да се реши като се извика конкретен `print()` или WorkingStudent генерира свой `print()`.

```

WorkingStudent ws;
ws.Worker::print(); // грешка
ws.Student::print(); // грешка
ws.print(); // грешка - нееднозначно е

```